Amlogic

NN Tool FAQ

Revision: 1.0

Release Date: 2024-07-17

Copyright

Trademarks

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision History

Version 1.0 (2024-07-17)

This is the first official release.

Contents

1. Framework

1.1 Which open-source frameworks are currently supported by Amlogic NN?

Answer: The currently supported neural network frameworks and their corresponding version information are as follows:

Framework	Version	Model load(Remark)
Tensorflow	2.15.0	Tf.compat.v1.lite.TFLiteConverter.from_frozen_graph()
Pytorch	2.0.0	torch.jit.load(model_path)
Onnx	1.12.0	onnx.load(model_path)
Mxnet	1.6.0	mx.model.load_checkpoint(prefix_path, prefix_epoch)
Darknet	https://github.com/siju-samuel/darknet	
Caffe	apt-get install caffe-cpu	caffe.Net(proto_file,blob_file,caffe.TEST)
tflite	2.15.0	tflite.Model.GetRootAsModel tflite.Model.Model.GetRootAsModel
Keras	3.0.5	tf.keras.models.load_model(model_path)
Paddle	2.4.2	Paddle.jit.load(model_path)

1.2 What model formats are currently supported by Amlogic NN?

Answer:Currently, Amlogic NN supports only quantized models. Models from supported open-source frameworks need to be quantized. The quantization methods include Int8, Int16, and Uint8, with support for both per-layer and per-channel quantization.

1.3 Which general models are currently supported by Amlogic NN?

Answer: The general models currently supported by Amlogic NN include SqueezeNet, VGG, ResNet*, Inception, RetinaNet*, MobileNet*, LSTM*, GRU*, SSD, Yolo*, Faster R-CNN, and Transformer*. It is recommended to use adla_tool directly for model conversion. All models that can generate adla files normally are supported.

1.4 Can multiple NN tasks run simultaneously on the NPU?

Answer: Multiple threads and processes are supported, but the underlying processing is done in a single-process, single-frame manner.

2. Tool

2.1 Is the quantization method of the Adla_tool consistent with TensorFlow's quantization method?

Answer: Yes, it is consistent. Currently, it supports three quantization methods: int8, int16, and uint8.

2.2 Error running the tool: xxx.so No such file or directory

adla-toolkit-binary-3.0.7.4/bin$ ./adla_convert
[13979] Error loading Python lib '/mnt/e/adla-toolkit-binary-3.0.7.4/bin/libpython3.10.so.1.0': dlopen: /mnt/e/adla-toolkit-binary-3.0.7.4/bin/libpython3.10.so.1.0: cannot open shared object file: No such file or directory

Answer: The tool needs to be used directly after being extracted in a Linux environment. After extraction, avoid copying or moving the folder in a directory manner.

3. Model Conversion

3.1 Will there be accuracy loss during model conversion?

Answer: Model conversion involves quantizing floating-point data to int8/uint8/int16, which may result in some degree of accuracy loss. Int16 per-channel quantization generally maintains the closest accuracy to the original floating-point precision. The quantization accuracy ranking is as follows: int8 per-layer < int8 per-channel < int16 per-channel.

3.2 During quantization, how many images are needed in the dataset? Is the quantity proportional to the accuracy?

Answer: It is generally recommended to use around 200 to 2000 images for the dataset, preferably images that reflect the scenarios in which the model will be used or images from the validation set of the training model. This has a positive effect on the quantization results. However, the quantity of images is not directly proportional to accuracy.

Recommendation: Use ./bin/adla_plug_in/tools/Quantize_optimization_tool to find the optimal quantization parameters.

3.3 When exporting case code using the adla_tool, how do you set the correct parameters for the corresponding board?

Answer: Execute the command: cat /proc/cpuinfo or cat /proc/cpu_chipid. Check the second-to-last line for the Serial number and use the first four characters of the serial (e.g., 3d0a) to determine the setting value based on the following correspondence table:

平台	PID	Serial
C3	0XA001	Serial: **3d0a**010100000000c613492931563343
S5	0XA002	Serial: **3e0a**0202000000007c404b6c31563553
T7C	0XA003	Serial: **360c**01030000000025f904ab31563541
T3X	0XA004	Serial: **420a**010100000000daff6e4931583354
S6	0XA005	Serial: **480a**0201000000006827632931563653

Example:

If the first four characters of the serial number are 3d0a, set the parameter as --target-platform PRODUCT_PID0XA001.

If the first four characters of the serial number are 3e0a, set the parameter as --target-platform PRODUCT_PID0XA002.

If the first four characters of the serial number are 360c, set the parameter as --target-platform PRODUCT_PID0XA003.

If the first four characters of the serial number are 420a, set the parameter as --target-platform PRODUCT_PID0XA004.

If the first four characters of the serial number are 480a, set the parameter as --target-platform PRODUCT_PID0XA005.

If the first four characters of the serial number are not listed in the above table, it is currently recommended to try using the parameter corresponding to the first three characters of the serial number.

3.4 Error during Caffe model conversion: Not supported caffenet model version (v0 layer or v1 layer)

Answer: The current Caffe model or prototxt file version is too old. You need to use a tool to update the prototxt or Caffe model file.

Solution:

To update the prototxt file, use the tool: upgrade_net_proto_text -d.

To update the caffemodel file, use the tool: upgrade_net_proto_binary -d.

How to Obtain: Compile the Caffe source code, and the corresponding executables will be generated in the tools directory.

3.5 Can the input data for quantization only be images? How should quantization data be provided if the model input data is not images?

Answer: Quantization data does not have to be images. You can save arrays or data from a txt file as an npy file, and then use the path to the npy file in dataset.txt instead of an image path during quantization. For example:

Use or refer to the acuity_tools_xxx/bin/adla_plug_in/tools/input_npy_txt_bin_generate.py script to save the npy file:

python3 input_npy_txt_bin_generate.py 1,3,224,224 npy np.float32 0 255

3.6 Common issues with Caffe models

Answer:

a、In a Caffe model, the format of the input layer typically includes the following aspects:

Error

```
name: "MobileNet-SSD"
input: "data"
input_shape {
  dim: 1
  dim: 3
  dim: 300
  dim: 300
}
layer {
  name: "conv0"
  type: "Convolution"
  bottom: "data"
  top: "conv0"
  param {
    lr_mult: 0.1
    decay_mult: 0.1
  }
```

Right

```
name: "MobileNet-SSD"
layer {
  name: "data"
  type: "Input"
  top: "data"
  input_param {
    shape {
      dim: 1
      dim: 3
      dim: 300
      dim: 300
    }
  }
}
layer {
  name: "conv0"
  type: "Convolution"
```

b、In prototxt files, unsupported layers can lead to conversion failures. For example, detection models that use the DetectionOutput layer are not supported. In such cases, you need to manually remove that layer from the prototxt file.

```
}
layer {
  name: "detection_out"
  type: "DetectionOutput"
  bottom: "mbox_loc"
  bottom: "mbox_conf_flatten"
  bottom: "mbox_priorbox"
  top: "detection_out"
  include {
    phase: TEST
  }
  detection_output_param {
    num_classes: 21
    share_location: true
    background_label_id: 0
    nms_param {
      nms_threshold: 0.45
      top_k: 100
    }
```

4. Integration

4.1 If an error occurs during model conversion, how can you identify the problem?

Answer: If the model conversion fails, you can set the environment variable export ADLA_LOG_LEVEL=DEBUG and then try the model conversion again. This will provide more detailed logging to help identify the problem.

(1)To determine at which step the error occurs:

In the conversion log, search for the keyword: Adla_tool. During the conversion process, the search results for this keyword can include the following types:

I Adla_tool: step_0 enter, begin to load original model.

I Adla_tool: step_0 out, load original model successfully.

I Adla_tool: step_1 enter, frontend begin to parse model.

I Adla_tool: step_1 out, frontend parse model successfully.

I Adla_tool: step_2 enter, begin to convert model to ir.

I Adla_tool: step_2 out, convert model to ir successfully.

I Adla_tool: step_3 enter, begin to quantize model.

I Adla_tool: step_3 out, quantize model successfully.

I Adla_tool: step_4 enter, begin to convert adla_file.

I Adla_tool: step_4 out, convert adla_file successfully,

By reviewing the process log, you can preliminarily determine at which step the exception occurred.

(2)Handling methods for errors occurring at different stages

Step_0: Model Loading Error:

- If the log after searching for the keyword only shows `step_0 enter`, it indicates that the original model failed to load. In this case, the customer should verify whether there is an issue with the original model by checking the versions and model loading methods for each framework as listed in Section 1.1 of the documentation.

Step_1: Error in Front-End Model Parsing:

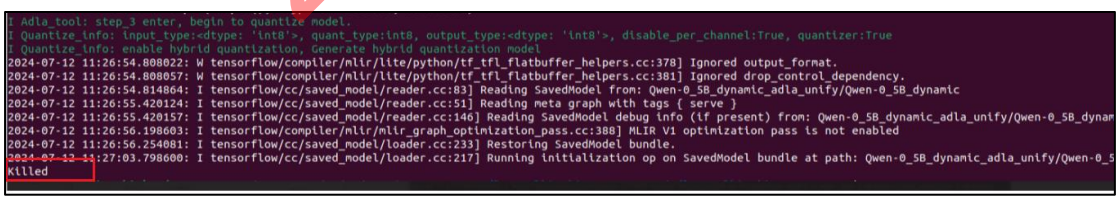
- Provide the model to Amlogic for debugging.

Step_2: Error During IR Conversion:

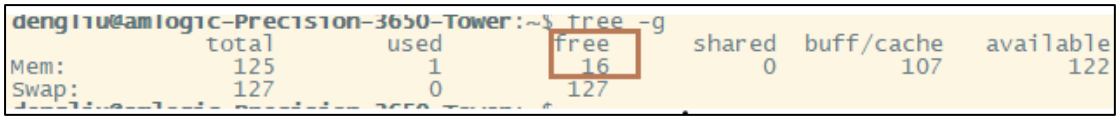
- First, check with Amlogic if the log message is clear. If the log information is insufficient to identify the issue, consider providing the model to Amlogic for debugging.

Step_3: Error During Model Quantization Conversion:

- Confirm whether the error log indicates a kill or other issues, such as:



If the error indicates kill, it may be due to insufficient server memory. It is recommended to check the available memory during model conversion using the command free -g.



Solution: Refer to https://blog.csdn.net/A308114/article/details/118544964 to increase the swap memory or consider upgrading the server.

Note: Increasing swap memory will use up system disk space. After model conversion, you can reduce the swap space back to its original size.

Step_4: Compiler Conversion Error:

- First, check with Amlogic to see if the log message is clear. If the log information is insufficient to pinpoint the issue, export the intermediate layers of the model conversion as a quantized TFLite file and send it to Amlogic for debugging.

Export Method: Before model conversion, set the environment variable export ADLA_EXPORT_MIDDLE_TO_TFLITE=True, and then perform the model conversion.

4.2 If the accuracy of the results on the board is poor, how can you diagnose the issue?

Answer:

(1)Confirm if the float results match the original model's accuracy:

The current tool supports exporting the IR during the model conversion process as a float TFLite model. First, verify if the exported float TFLite results are consistent with the original model's results.

Export Method:

a. Set the environment variable: export ADLA_EXPORT_MIDDLE_TO_TFLITE=True.
b. During model conversion, choose the quantization type as float32, e.g., --quantize-dtype float32.
c. The output directory will then contain the float TFLite model, which can be compared with the original model's results.
d. Note: If the original model is in nchw format, ensure that the same preprocessing is used when testing the float TFLite model, and then perform a transpose operation if needed.

(2)Confirm if the float results and quantized results are consistent:

Export IR during model conversion to both float TFLite and quantized TFLite, and check if the float results match the quantized results.

Quantized Export Method:

a. Set the environment variable: export ADLA_EXPORT_MIDDLE_TO_TFLITE=True.
b. Perform the model quantization conversion as usual, save the quantized files, and generate the quantized TFLite.

(3)Confirm if the quantized results match the board-side results:

Compare the PC-side quantized TFLite results with the board-side results, ensuring input consistency. The output format on the board is NHWC, the same as TFLite. You can use the demo_adla_compare_buf.py script in NNIDE_Tool to compare the quantized TFLite and board-side results.

Problem Analysis:

a. **If the first step results are abnormal:**

After confirming input consistency, if the results are still inconsistent, it indicates that adla_tool may have issues with model conversion adaptation. Provide the model, quantization images, and quantization parameter information to Amlogic for debugging.

b. **If the second step results are abnormal:**

If the first step results are okay but the second step results are abnormal, it indicates a quantization accuracy issue. You need to check the following:

- Verify if the quantization parameters, such as channel-mean-value, are correct. Ensure that the dataset provided in dataset.txt has sufficient images or npy files, generally recommended to be 200–500.
- Accuracy ranking of the three quantization methods: int8 perlayer < int8 perchannel < int16 perchannel. If the quantization input data and channel-mean-value are correct, consider trying int8 perchannel or int16 perchannel quantization.
- Confirm which quantization method meets the accuracy requirements. If all methods have accuracy issues, provide the model, quantization images, and quantization parameter information to Amlogic for debugging.

c. If the third step results are abnormal:

- If the second step results are normal but the third step results are incorrect, it suggests an issue with the underlying driver or compiler. Try the following:
- Before converting the model, set the environment variable: export ADLA_PRECISE_MODEL=True, generate a new adla file, and test it on the board.
- If the results are normal, it indicates that the issue may be with the compiler during the conversion and fusion process. You can use this method to avoid accuracy issues. If results are still incorrect, it may indicate a bug in the driver. Provide the generated quantized TFLite to Amlogic for debugging.

4.3 After integrating the demo, if the preprocessing time is long and is the bottleneck in the entire process, how can this be resolved?

Answer:

Demo Preprocessing Steps: a. Preprocess the original data `data0` to obtain `data1` (e.g., for classification models, normalize the image after reading). b. Quantize the preprocessed data `data1` to obtain quantized data `data2`. c. Send the quantized data `data2` to the NPU for inference.

Solution 1: Try to merge the first two preprocessing steps

For classification models, for example:

- **Original Data (`data0`):** Image data with a range of 0–255.
- **Preprocessed Data (`data1`):** Normalize data0 to get data1, e.g., data1 = data0 / 255 (range 0–1) or data1 = (data0 - 128) / 128 (range -1 to 1).
- **Quantized Data (`data2`):** Quantize data1 to get data2, where data2 = data1 / scale - zero_point.

Since the preprocessing operations and values for scale and zero_point are known, you can directly compute data2 from data0 by substituting the constants into the formula. This saves preprocessing and quantization time. For some models, this results in data2 = (uint8) data0 or data2 = (int8) (data0 - 128). For non-image models, you can also use this method to check if it saves preprocessing and quantization time.

Retrieve Scale and Zero Point:

- After converting to ADLA, you can obtain the scale and zero point using adlainfo:/xxxx/bin/adla_plug_in/tools/adla_info_tool/adlainfo_release -io xxxx.adla

For example, for ResNet, you might find input scale: 0.007782 and zero_point:1.

```
adla file name : caffe_output/resnet-18_int8.adla

****************************************
    Inputs                    : [0 ]
        Input[0]              : 0
        Dim Count             : 4
        Size of dim[0]        : 1
        Size of dim[1]        : 224
        Size of dim[2]        : 224
        Size of dim[3]        : 3
        type                  : Int8
        scale                 : [0.007782 ]
        zero_point            : [1 ]
        inputs_memory_size    : 147.00 kb
****************************************

****************************************
    Outputs                   : [96 ]
        Output[0]             : 96
        Dim Count             : 2
        Size of dim[0]        : 1
        Size of dim[1]        : 1000
        type                  : Int8
        scale                 : [0.003906 ]
        zero_point            : [-128 ]
        outputs_memory_size   : 0.98 kb
****************************************
    config:
      max_outstanding_outputs : 4
      axi_sram_size           : 262144 bytes
****************************************
```


Solution 2: Improve the Efficiency of Step 3

- Typically, the aml_module_input_set interface is used to send quantized data data2 to the NPU for inference. Currently, you can refer to the ADLA_SDK_API documentation to call the aml_util_mallocBuffer interface to allocate input/output buffers and set the input/output format to INPUT_DMA_DATA.
- During preprocessing, write the processed data directly into the buffer allocated by the aml_util_mallocBuffer interface. This can save the time it takes to copy data from DDR to the NPU.

4.4 After integrating the demo, if the dequantization time during post-processing is long, how can this be resolved?

Answer: The NPU only supports fixed-point data, and float data is used for post-processing. Generally, after obtaining the NPU results, the fixed-point data needs to be dequantized to float before post-processing. However, if the output buffer is large, the dequantization operation can be time-consuming.

Current Suggested Solution: Directly process the fixed-point data first and then selectively perform dequantization.

Example 1: For Classification Models

- If the post-processing involves a top-5 operation, you can first obtain the top-1 result based on the fixed-point data and then only dequantize the top-1 result to float.
- Normally, you would need to dequantize [1,1000] data points, but with this optimization, only one data point needs to be dequantized.

Example 2: For Detection Models (e.g., YOLOv5)

- If the post-processing involves non-maximum suppression (NMS) and other operations, you can first convert the NMS threshold to a fixed-point value `threshold1`.
- Use `threshold1` to process the fixed-point data output by the NPU. After filtering the data with `threshold1`, perform dequantization on the filtered data.

4.5 How to obtain the result and execution time of each operation?

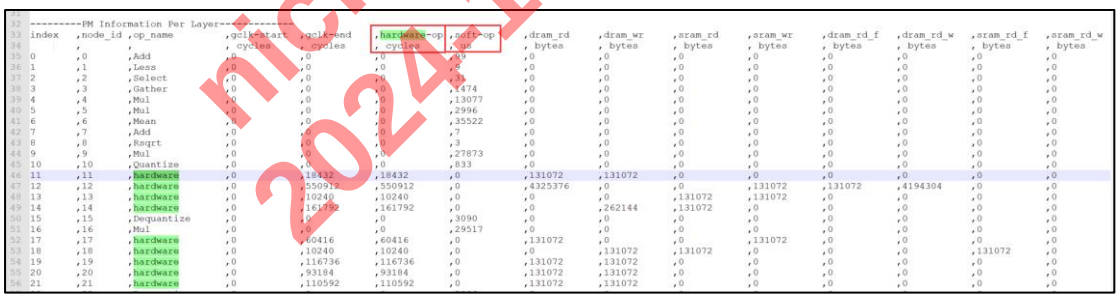
Answer: When running the model on the board, set the environment variables:

```
export ADLA_ENABLE_DEBUG_AND_PROFILING=1
export ADLA_INVOKE_PER_LAYER=1
export ADLA_DEBUG_OUTPUT_PATH=/data/test
```

The intermediate layer results are saved in /data/test/output, and profile_info.txt is saved in /data/test.

Analysis of profile_info.txt:

- **Hardware op:** The computation method is [hardware-op(cycles)] / 800 (us).
- **Soft op**: Corresponds to soft-op (us).



index	node_id	op_name	qclk-start, cycles	qclk-end, cycles	hardware-op, cycles	soft-op, us	dram_rd, bytes	dram_wr, bytes	sram_rd, bytes	sram_wr, bytes	dram_rd_f, bytes	dram_rd_w, bytes	sram_rd_f, bytes	sram_rd_w, bytes
0	0	Add	0	0	0	49	0	0	0	0	0	0	0	0
1	1	Less	0	0	0	2	0	0	0	0	0	0	0	0
2	2	Select	0	0	0	3	0	0	0	0	0	0	0	0
3	3	Gather	0	0	0	1474	0	0	0	0	0	0	0	0
4	4	Mul	0	0	0	13077	0	0	0	0	0	0	0	0
5	5	Mul	0	0	0	2996	0	0	0	0	0	0	0	0
6	6	Mean	0	0	0	35522	0	0	0	0	0	0	0	0
7	7	Add	0	0	0	7	0	0	0	0	0	0	0	0
8	8	Rsqrt	0	0	0	3	0	0	0	0	0	0	0	0
9	9	Mul	0	0	0	27873	0	0	0	0	0	0	0	0
10	10	Quantize	0	0	0	833	0	0	0	0	0	0	0	0
11	11	hardware	0	18432	18432	0	131072	131072	0	0	0	0	0	0
12	12	hardware	0	550912	550912	0	4325376	0	0	131072	131072	4194304	0	0
13	13	hardware	0	10240	10240	0	0	0	131072	131072	0	0	0	0
14	14	hardware	0	161792	161792	0	0	262144	131072	0	0	0	0	0
15	15	Dequantize	0	0	0	3090	0	0	0	0	0	0	0	0
16	16	Mul	0	0	0	29517	0	0	0	0	0	0	0	0
17	17	hardware	0	60416	60416	0	131072	0	0	131072	0	0	0	0
18	18	hardware	0	10240	10240	0	0	131072	131072	0	0	0	131072	0
19	19	hardware	0	116736	116736	0	131072	131072	0	0	0	0	0	0
20	20	hardware	0	93184	93184	0	131072	131072	0	0	0	0	0	0
21	21	hardware	0	110592	110592	0	131072	131072	0	0	0	0	0	0

Note: The current hardware does not display the corresponding OP names. It is expected to be fixed in the next version.

4.6 How to check the utilization of the NPU?

Answer:

For Android environment:

1.1: echo 1 > /sys/class/adla/adla0/device/debug/utilization

1.2: cat /sys/class/adla/adla0/device/debug/utilization

For Linux environment:

1.3: cat /proc/mbp/adla_utilization

	Possible phenomena	Note
cat utilization	please [echo 1 > utilization] first	If the monitoring switch is not turned on, proceed to step 1.1 first
	adla utilization : -1%	Currently adla is not performing computation, either because the task queue has no task, or it is in a task scheduling state
	adla utilization : xx%	xx is a non--1 value, and the xx obtained at this time is the real-time utilization of adla

4.7 How do you get the bandwidth of a model run?

Answer: After model transformation using adla_tool, a case demo is generated. For now, you just need to add aml_util_enableProfile() and aml_util_getProfileInfo() calls to the case demo. Refer to Section 3.5 of the ADLA_SDK_API documentation, or to the following code:

invoke_time: npu takes time

BW(Ddr_data) : Bandwidth usage

memory_usage_bytes:Memory consumption

```
context = aml_module_create(&config);

aml_profile_config_t profile_data;
aml_util_enableProfile(context, &profile_data);
ret = run_network(netType, context, jpath);
aml_util_getProfileInfo(context, &profile_data);

fprintf(stdout, "\nProfilling data:\n");
fprintf(stdout, "model_name: %s\n", config.path);

fprintf(stdout, "  -------------From system-----------------\n");
fprintf(stdout, "  core_freq_cur         : %llu Hz\n", profile_data.profiling_data.ext.core_freq_cur);
fprintf(stdout, "  axi_freq_cur          : %llu Hz\n", profile_data.profiling_data.ext.axi_freq_cur);
fprintf(stdout, "  mem_pool_size         : 0x%08llX Bytes\n", profile_data.profiling_data.ext.mem_pool_size);
fprintf(stdout, "  mem_pool_used         : 0x%08llX Bytes\n", profile_data.profiling_data.ext.mem_pool_used);
fprintf(stdout, "  mem_alloced_base      : 0x%08llX Bytes\n", profile_data.profiling_data.ext.mem_alloced_base);
fprintf(stdout, "  mem_alloced_umd       : 0x%08llX Bytes\n", profile_data.profiling_data.ext.mem_alloced_umd);
int elapsed_hw_sw = (int)profile_data.profiling_data.ext.us_elapsed_in_hw_op + (int)profile_data.profiling_data.ext.us_elapsed_in_sw_op;

fprintf(stdout, "----------------------------aml need--------------\n");
fprintf(stdout, "  : %7.2f ms, inference_time \n", (float)profile_data.profiling_data.inference_time_us / 1000);
fprintf(stdout, "  : %7.2f ms, elapsed_in_hw_op  \n", (float)profile_data.profiling_data.ext.us_elapsed_in_hw_op / 1000);
fprintf(stdout, "  : %7.2f ms, elapsed_in_sw_op  \n", (float)profile_data.profiling_data.ext.us_elapsed_in_sw_op / 1000);
fprintf(stdout, "  : %7.2f ms, elapsed (hw_op+sw_op)  \n", (float)elapsed_hw_sw/1000);
fprintf(stdout, "  : %7.2f ms, invoke_time           \n", (float)usVal_inference / 1000);
fprintf(stdout, "  : %7.2f  FPS(pm)       \n",(double)1000000 / (double)profile_data.profiling_data.inference_time_us);
fprintf(stdout, "  : %7.2f  FPS(hw+sw)    \n", (double)1000000 / (double)(elapsed_hw_sw));
fprintf(stdout, "  : %7.2f  FPS(invoke) \n",(double)1000000 / (double)(usVal_inference));
fprintf(stdout, "  : %7.2f Mbytes,   dram_read_size \n",(float)profile_data.profiling_data.dram_read_bytes / (1024 * 1024));
fprintf(stdout, "  : %7.2f Mbytes,   dram_write_size \n",(float)profile_data.profiling_data.dram_write_bytes / (1024 * 1024));
fprintf(stdout, "  : %7.2f Mbytes,   BW(Ddr_data) \n",(double)(profile_data.profiling_data.dram_read_bytes + profile_data.profiling_data.dram_write_bytes) /  (
1024 * 1024));

fprintf(stdout, "  : %7.2f Mbytes,   sram_read_bytes \n",(float)profile_data.profiling_data.sram_read_bytes / (1024 * 1024));
fprintf(stdout, "  : %7.2f Mbytes,   sram_write_bytes \n",(float)profile_data.profiling_data.sram_write_bytes / (1024 * 1024));
fprintf(stdout, "  : %7.2f Mbytes,   BW(Sram_data) \n",(double)(profile_data.profiling_data.sram_read_bytes + profile_data.profiling_data.sram_write_bytes) /
        (1024 * 1024));

fprintf(stdout, "  : %7.2f Mbytes,   memory_usage_bytes \n",(float)profile_data.profiling_data.memory_usage_bytes / (1024 * 1024));

aml_util_disableProfile(context, &profile_data);
```